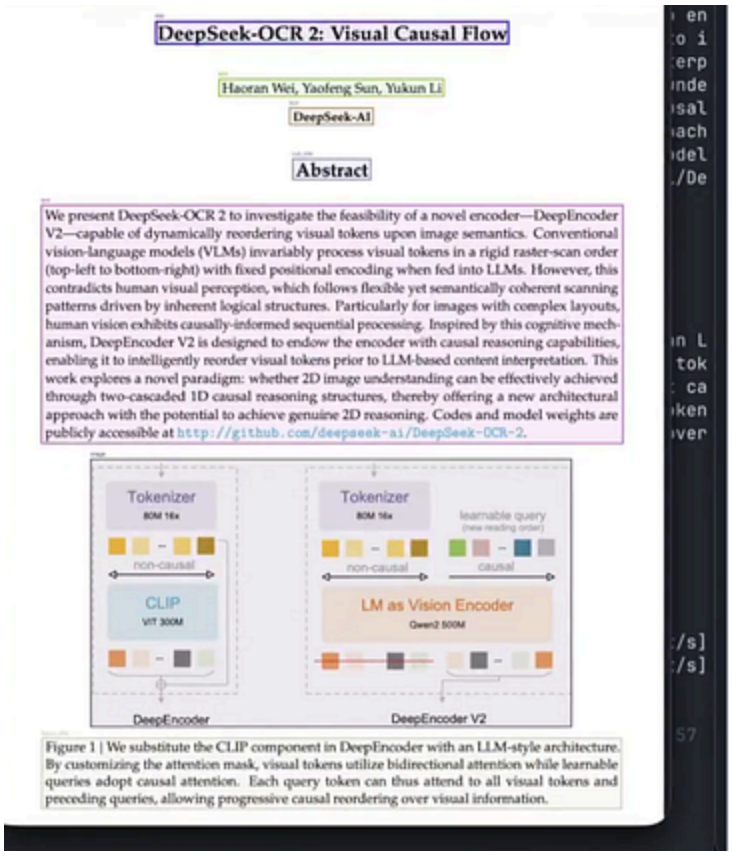
Milestone: 20GB Reached

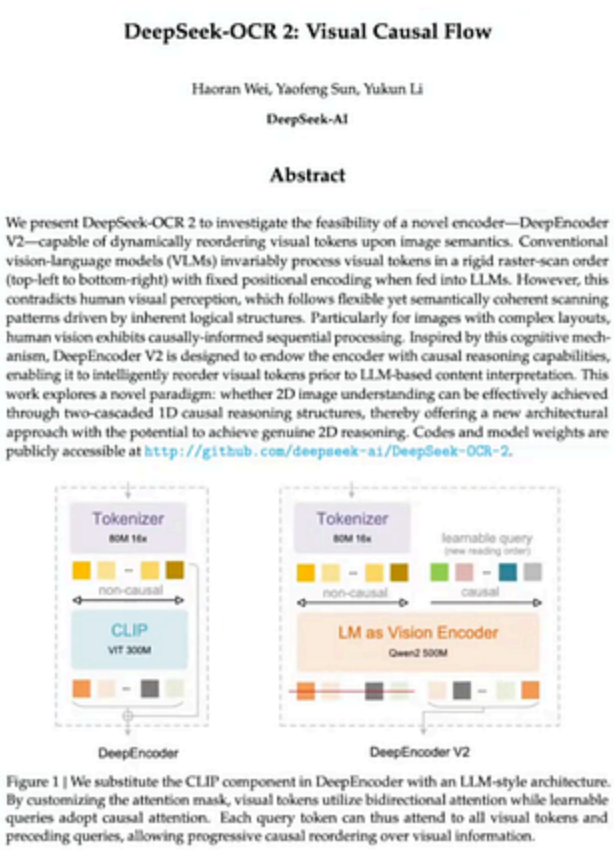
We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original





"This PDF hates OCR"—DeepSeek OCR 2 Reads Documents Like A Human

Your customer forwards an invoice PDF with the subject line: "why is this invoice scrambled?"

Agent Native Follow
~9 min read · February 2, 2026 (Updated: February 2, 2026) · Free: No

The painful part is the text is usually there. The model just can't keep the **structure** together.

It reads a table like random paragraphs. It merges columns. It separates labels from values. And no amount of regex duct tape really fixes it.

OCR pipelines have **nasty long tails in production**: PDFs with tables, multi-column layouts, mixed fonts, screenshots, mobile photos, math, stamps, weird spacing, skew, poor lighting, tiny footnotes..

That's why I pay attention when a model claims it can **read documents like humans do**.

DeepSeek-OCR 2 is one of those releases that's worth a real evaluation, mostly because its core idea maps onto the failure modes I see in production:

- layout breaks (wrong reading order)
- tables flatten incorrectly
- labels get separated from values
- multi-column flows collapse into nonsense

DeepSeek OCR 2 is trying to fix that at the encoder level.

If you haven't, please also grab the first issue of *Agentic SaaS Patterns Winning in 2026*.

What DeepSeek OCR 2 actually is

At a high level, DeepSeek OCR 2 is an **open-source, Apache-2.0 licensed**, image-to-text model that targets **document understanding** (not just transcription).

It's published with weights + code and runs via standard Hugging Face Transformers (with `trust_remote_code=True`).

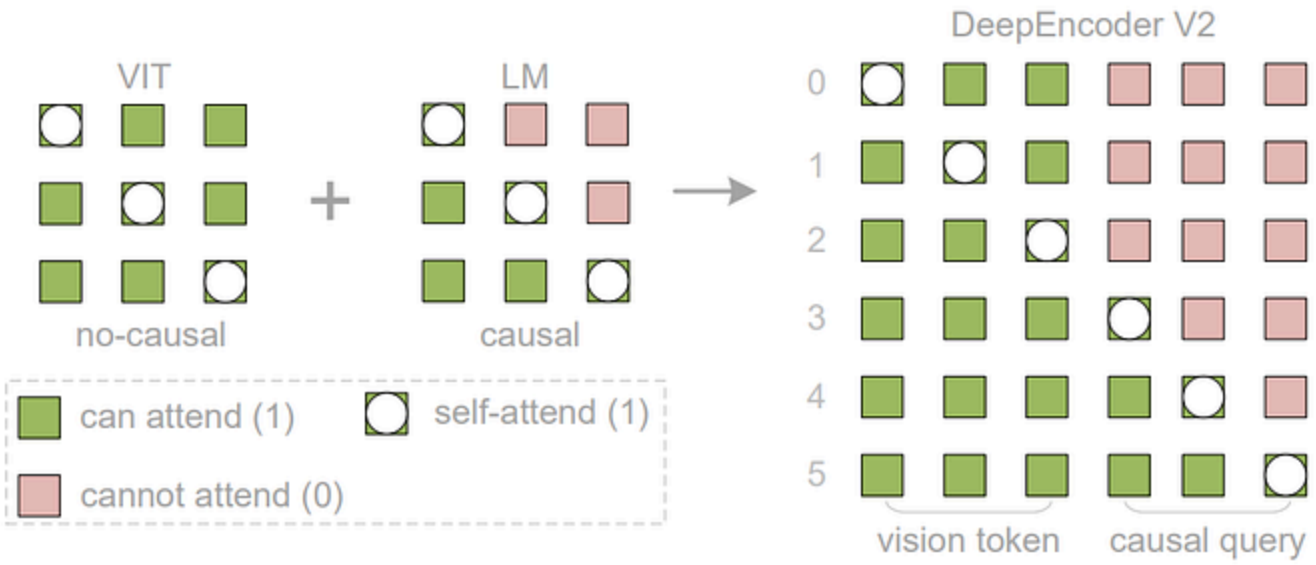
A few concrete facts that matter immediately if you build products:

- **3B params**, released in BF16.
- The main checkpoint (`model-00001-of-000001.safetensors`) is **~6.78GB**.
- Official docs emphasize **NVIDIA CUDA** setups (e.g., CUDA 11.8 + Torch 2.6.0).
- It includes a production-ish path using **vLLM** with scripts for **streaming image OCR**, **concurrent PDF OCR**, and **batch eval**.

So the "it's real" box is checked: license, weights, code, and an inference story beyond a single notebook cell.

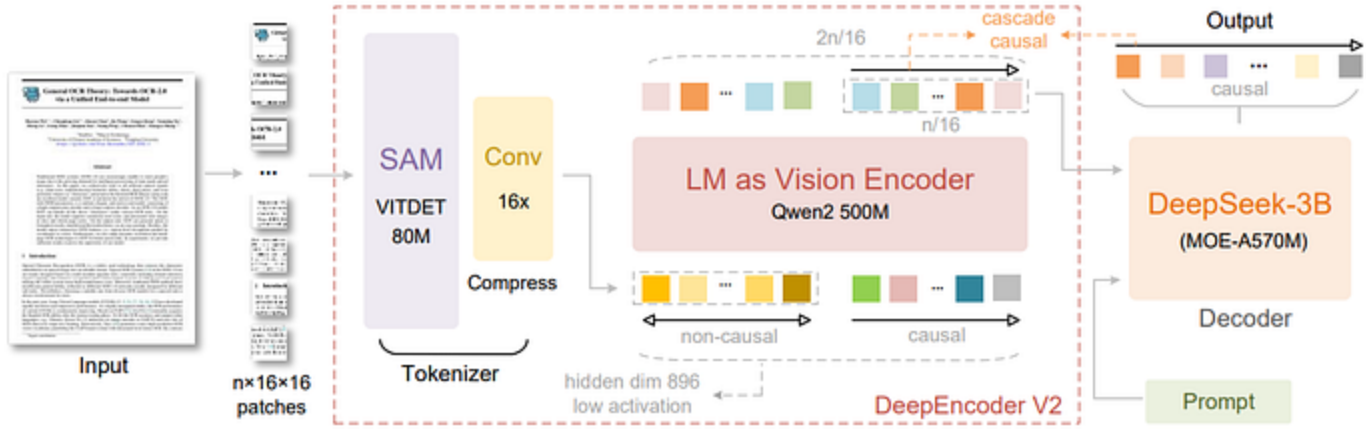
What "Visual Causal Flow" Really Means

DeepEncoder V2 is replacing the CLIP component used previously with a compact LLM-style encoder, it's using **learnable "causal flow" queries** and applying a customized attention mask where visual tokens keep bidirectional attention, but query tokens operate causally, feeding only the query outputs (the "reordered" side) into the decoder.



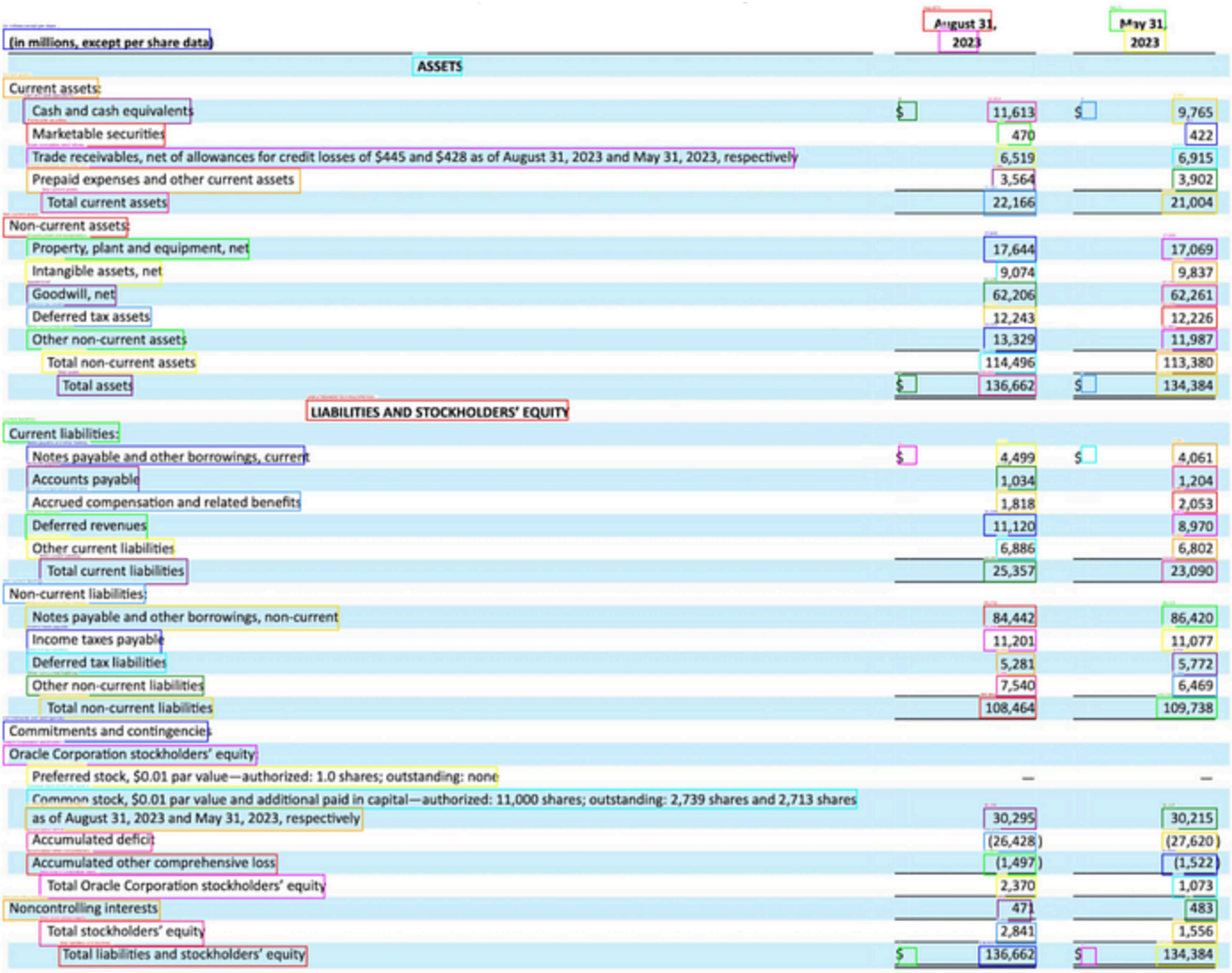
There's also some concrete implementation detail that gives this credibility.

The vision tokenizer uses an 80M-parameter SAM-base component + conv layers (following their earlier approach), primarily to achieve large token compression while keeping compute manageable.



I like seeing this level of detail because it helps me reason about:

- what's likely to break (edge-case layouts),
- what's likely to generalize (reading flow),
- where fine-tuning might help (domain layouts).



(in millions, except per share data)	August 31, 2023	May 31, 2023
ASSETS		
Current assets:		
Cash and cash equivalents	$ 11,613	$ 9,765
Marketable securities	470	422
Trade receivables, net of allowances for credit losses of $445 and $428 as of August 31, 2023 and May 31, 2023, respectively	6,519	6,915
Prepaid expenses and other current assets	3,564	3,902
Total current assets	22,166	21,004
Non-current assets:		
Property, plant and equipment, net	17,644	17,069
Intangible assets, net	9,074	9,837
Goodwill, net	62,206	62,261
Deferred tax assets	12,243	12,226
Other non-current assets	13,329	11,987
Total non-current assets	114,496	113,380
Total assets	$ 136,662	$ 134,384
LIABILITIES AND STOCKHOLDERS' EQUITY		
Current liabilities:		
Notes payable and other borrowings, current	$ 4,499	$ 4,061
Accounts payable	1,034	1,204
Accrued compensation and related benefits	1,818	2,053
Deferred revenues	11,120	8,970
Other current liabilities	6,886	6,802
Total current liabilities	25,357	23,090
Non-current liabilities:		
Notes payable and other borrowings, non-current	84,442	86,420
Income taxes payable	11,201	11,077
Deferred tax liabilities	5,281	5,772
Other non-current liabilities	7,540	6,469
Total non-current liabilities	108,464	109,738
Commitments and contingencies		
Oracle Corporation stockholders' equity:		
Preferred stock, $0.01 par value—authorized: 1.0 shares; outstanding: none	—	—
Common stock, $0.01 par value and additional paid in capital—authorized: 11,000 shares; outstanding: 2,739 shares and 2,713 shares as of August 31, 2023 and May 31, 2023, respectively	30,295	30,215
Accumulated deficit	(26,428)	(27,620)
Accumulated other comprehensive loss	(1,497)	(1,522)
Total Oracle Corporation stockholders' equity	2,370	1,073
Noncontrolling interests	471	483
Total stockholders' equity	2,841	1,556
Total liabilities and stockholders' equity	$ 136,662	$ 134,384

The most interesting part: OCR 2 learns a reading path

Most VLM-ish OCR pipelines effectively treat a page like a **fixed 1D sequence**: chunk into patches, then serialize left-to-right, top-to-bottom.

DeepSeek's paper calls this a **rigid raster-scan order** and points out it contradicts how humans visually process complex layouts.

That raster assumption is fine for "one column of text on a clean background."

It's conceptually broken for real documents:

- a table in the bottom-left gets "sliced" across the token sequence
- form fields get interleaved with surrounding text
- multi-column reading order becomes a guessing game
- diagrams + captions lose locality

DeepSeek OCR 2's approach is closer to **encode everything globally, then learn *how to traverse* it.**

Step 1: bidirectional "what's on the page" tokens The encoder builds a representation of the whole image. The model's motivation is to avoid forcing a hard-coded scan order when semantics live in 2D.

Step 2: learnable "query" tokens that decide what to read next Instead of committing to a fixed traversal, DeepSeek OCR 2 introduces what the paper frames as *causal flow* — a mechanism to dynamically reorder visual tokens based on image semantics before decoding.

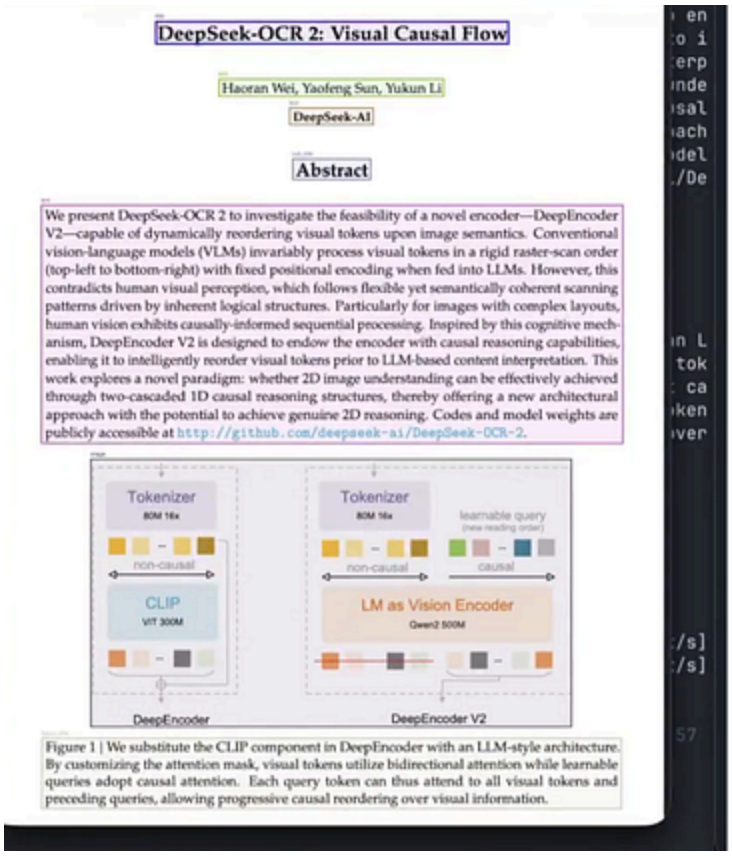
The intuition:

- The bidirectional tokens represent **the page**.
- The causal/query side represents **a learned reading plan**.

If the bottom-left is a table, the model can learn to "camp there" and output it as a contiguous block, rather than scattering it across the output because the raster order forced it.

It's basically saying *reading order is a model capability, not a heuristic.*

And if you've ever tried to patch reading order downstream with brittle rules… you immediately understand why this matters.



DeepSeek-OCR 2: Visual Causal Flow

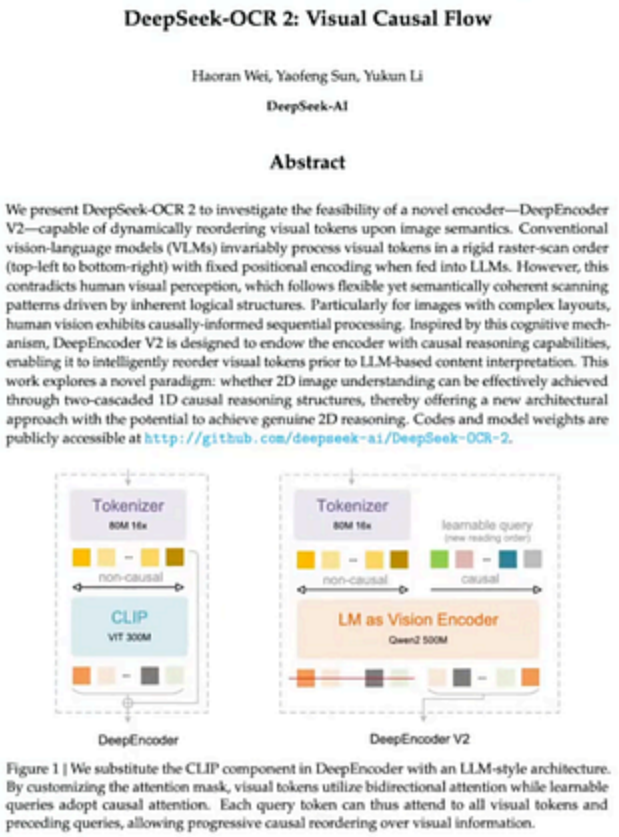
Haoran Wei, Yaofeng Sun, Yukun Li

DeepSeek-AI

Abstract

We present DeepSeek-OCR 2 to investigate the feasibility of a novel encoder—DeepEncoder V2—capable of dynamically reordering visual tokens upon image semantics. Conventional vision-language models (VLMs) invariably process visual tokens in a rigid raster-scan order (top-left to bottom-right) with fixed positional encoding when fed into LLMs. However, this contradicts human visual perception, which follows flexible yet semantically coherent scanning patterns driven by inherent logical structures. Particularly for images with complex layouts, human vision exhibits causally-informed sequential processing. Inspired by this cognitive mechanism, DeepEncoder V2 is designed to endow the encoder with causal reasoning capabilities, enabling it to intelligently reorder visual tokens prior to LLM-based content interpretation. This work explores a novel paradigm: whether 2D image understanding can be effectively achieved through two-cascaded 1D causal reasoning structures, thereby offering a new architectural approach with the potential to achieve genuine 2D reasoning. Codes and model weights are publicly accessible at http://github.com/deepseek-ai/DeepSeek-OCR-2.

Figure 1 | We substitute the CLIP component in DeepEncoder with an LLM-style architecture. By customizing the attention mask, visual tokens utilize bidirectional attention while learnable queries adopt causal attention. Each query token can thus attend to all visual tokens and preceding queries, allowing progressive causal reordering over visual information.



DeepSeek-OCR 2: Visual Causal Flow

Haoran Wei, Yaofeng Sun, Yukun Li

DeepSeek-AI

Abstract

We present DeepSeek-OCR 2 to investigate the feasibility of a novel encoder—DeepEncoder V2—capable of dynamically reordering visual tokens upon image semantics. Conventional vision-language models (VLMs) invariably process visual tokens in a rigid raster-scan order (top-left to bottom-right) with fixed positional encoding when fed into LLMs. However, this contradicts human visual perception, which follows flexible yet semantically coherent scanning patterns driven by inherent logical structures. Particularly for images with complex layouts, human vision exhibits causally-informed sequential processing. Inspired by this cognitive mechanism, DeepEncoder V2 is designed to endow the encoder with causal reasoning capabilities, enabling it to intelligently reorder visual tokens prior to LLM-based content interpretation. This work explores a novel paradigm: whether 2D image understanding can be effectively achieved through two-cascaded 1D causal reasoning structures, thereby offering a new architectural approach with the potential to achieve genuine 2D reasoning. Codes and model weights are publicly accessible at http://github.com/deepseek-ai/DeepSeek-OCR-2.

Figure 1 | We substitute the CLIP component in DeepEncoder with an LLM-style architecture. By customizing the attention mask, visual tokens utilize bidirectional attention while learnable queries adopt causal attention. Each query token can thus attend to all visual tokens and preceding queries, allowing progressive causal reordering over visual information.

Token budgets and "reading order" are the whole game

A lot of OCR model comparisons obsess over accuracy without talking about economics. In production OCR, token budget translates to:

- latency
- cost
- throughput
- max page complexity you can support

DeepSeek OCR 2's benchmark framing is unusually crisp because they talk in **visual tokens per page**.

It constrains visual tokens fed into the LLM decoder in the **256-1120** range, and the paper explicitly compares **Gemini-3 Pro at 1120** visual tokens in OmniDocBench v1.5 edit distance results.

Also, the default "dynamic resolution" scheme is documented as:

- **(0–6) x 768 x 768 local views + 1 x 1024 x 1024 global view**
- mapped to visual tokens: **(0–6)×144 + 256**

So your "quality vs speed" knob is basically **how many local crops are you willing to pay for?**

What changed from DeepSeek-OCR (V1) to V2

DeepSeek OCR (v1) was already notable for pushing the idea of **optical compression**, i.e. keep vision tokens low, decode long text outputs anyway.

Their v1 paper reports:

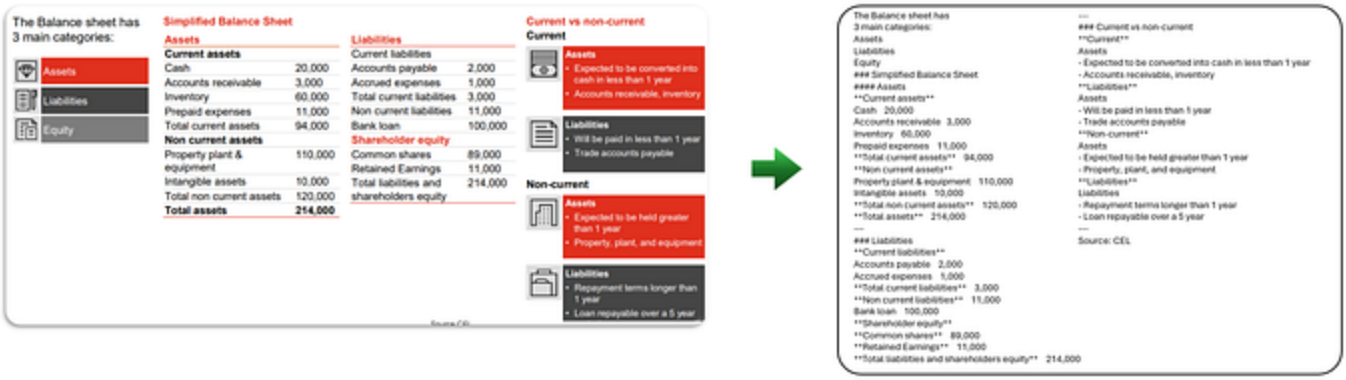
- at **<10x compression ratio**, OCR precision can hit **97%**
- at **20x**, accuracy still around **60%**
- and a production claim of **200k+ pages/day** on a single **A100–40GB** for training data generation

OCR 2 keeps the "compression + decode" framing, but the benchmark gains are attributed to the new encoder design.

On **OmniDocBench v1.5**, the OCR 2 paper reports:

- **91.09% overall** at **V-tokenmax = 1120**
- **+3.73%** improvement vs DeepSeek OCR baseline under similar data sources
- reading-order edit distance improvement: **0.085 to 0.057**

That last line is the one I care about, because reading order is exactly where "OCR is solved" dies in real products.



The benchmark comparison that will drive half the discourse

DeepSeek OCR 2 includes a direct OmniDocBench v1.5 edit distance table that compares it to Gemini-3 Pro under the **same** max visual token budget:

- **Gemini-3 Pro** @ 1120: **OverallEdit = 0.115**
- **DeepSeek OCR 2** @ 1120: **OverallEdit = 0.100**

DeepSeek-OCR 2 reports **+3.73% performance gains on OmniDocBench v1.5** vs the DeepSeek-OCR baseline.

Model	V-token^{max} ↓	Overall ↑	Text^{Edit} ↓	Formula^{CDM} ↑	Table^{TEDs} ↑	Table^{TEDS_s} ↑	R-order^{Edit} ↑
Pipline							
Marker-1.8.2 [1]	-	71.30	0.206	76.66	57.88	71.17	0.250
MinerU2-pp [45]	-	71.51	0.209	76.55	70.90	79.11	0.225
Dolphin [17]	-	74.67	0.125	67.85	68.70	77.77	0.124
Dolphin-1.5 [17]	-	83.21	0.092	80.78	78.06	84.10	0.080
PP-StructureV3 [13]	-	86.73	0.073	85.79	81.68	89.48	0.073
MonkeyOCR-pro-1.2B [23]	-	86.96	0.084	85.02	84.24	89.02	0.130
MonkeyOCR-3B [23]	-	87.13	0.075	87.45	81.39	85.92	0.129
MonkeyOCR-pro-3B [23]	-	88.85	0.075	87.25	86.78	90.63	0.128
MinerU2.5 [45]	-	90.67	0.047	88.46	88.22	92.38	0.044
PaddleOCR-VL [12]	-	92.86	0.035	91.22	90.89	94.76	0.043
End-to-end Model							
OCRFlux [4]	>6000	74.82	0.193	68.03	75.75	80.23	0.202
GPT-4o [33]	-	75.02	0.217	79.70	67.07	76.09	0.148
InternVL3 [55]	>7000	80.33	0.131	83.42	70.64	77.74	0.113
POINTS-Reader [31]	>6000	80.98	0.134	79.20	77.13	81.66	0.145
olmOCR [36]	>6000	81.79	0.096	86.04	68.92	74.77	0.121
InternVL3.5-241B [49]	>7000	82.67	0.142	87.23	75.00	81.28	0.125
MinerU2-VLM [45]	>7000	85.56	0.078	80.95	83.54	87.66	0.086
Nanonets-OCR-s [2]	>7000	85.59	0.093	85.90	80.14	85.57	0.108
Qwen2.5-VL-72B [9]	>6000	87.02	0.094	88.27	82.15	86.22	0.102
Gemini-2.5 Pro[6]	-	88.03	0.075	85.82	85.71	90.29	0.097
dots.ocr [39]	>6000	88.41	0.048	83.22	86.78	90.62	0.053
OCRVerse [3]	>6000	88.56	0.058	86.91	84.55	88.45	0.071
Qwen3-VL-235B [8]	>6000	89.15	0.069	88.14	86.21	90.55	0.068
DeepSeek-OCR (9-crops)	1156	87.36	0.073	84.14	85.25	89.01	0.085
DeepSeek-OCR 2	1120	91.09	0.048	90.31	87.75	92.06	0.057
	↓ 36	↑ 3.73	↓ 0.025	↑ 6.17	↑ 2.5	↑ 3.05	↓ 0.028

On the Google DeepMind side, OmniDocBench 1.5 is reported as **Overall Edit Distance (lower is better)**. Gemini 3 Pro is shown at **0.115** in their published benchmark table, and their methodology PDF explicitly describes how OmniDocBench 1.5 results are averaged.

In DeepSeek's paper,

Model	V-token^{max}↓	Text^{Edit}↓	Formula^{Edit}↓	Table^{Edit}↓	R-order^{Edit}↓	Overall^{Edit}↓
Gemini-3 pro [44]	1120	-	-	-	-	0.115
Seed-1.8 [41]	5120	-	-	-	-	0.106
DeepSeek-OCR	1156	0.073	0.236	0.123	0.085	0.129
DeepSeek-OCR 2	1120	0.048	0.198	0.096	0.057	0.100

Edit Distances for different categories of document-elements in OmniDocBench v1.5

> DeepSeek-OCR 2 (0.100) achieves lower ED in document parsing compared to Gemini-3 Pro (0.115) under a similar visual token budget (1120), further demonstrating that our new model maintains high compression rates of visual tokens while ensuring superior performance, with exceptionally high potential.

Also interesting: OCR 2's breakdown in that same table is explicit for text/formula/table/reading order edit components (with OCR 2 improving reading order and tables relative to OCR 1).

The right way to read this is not "model X beats model Y forever." It's:

> *at a sane-ish visual token budget, OCR 2 is competitive with top-tier systems on document parsing metrics.*

And that's a big deal for an open-weight model.

How to Run DeepSeek-OCR 2 locally

I'll show you the most direct path first (Transformers), then the "I'm shipping this" path (vLLM batch + PDF).

Prereqs:

- python 3.12.9
- CUDA 11.8
- torch 2.6.0
- transformers 4.46.3
- flash-attn 2.7.3 (installed with `--no-build-isolation`)

Please note that the weight file is large (single shard `model-00001-of-000001.safetensors` ~6.78GB), so your first pull won't be subtle.

Transformers inference

Copy

```
from transformers import AutoModel, AutoTokenizer
import torch, os

os.environ["CUDA_VISIBLE_DEVICES"] = "0"
model_name = "deepseek-ai/DeepSeek-OCR-2"

tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
model = AutoModel.from_pretrained(
    model_name,
    _attn_implementation="flash_attention_2",
    trust_remote_code=True,
    use_safetensors=True,
)
model = model.eval().cuda().to(torch.bfloat16)

# prompt = "<image>\nFree OCR. "
prompt = "<image>\n<|grounding|>Convert the document to markdown. "
res = model.infer(
    tokenizer,
    prompt=prompt,
    image_file="your_image.jpg",
    output_path="your/output/dir",
    base_size=1024,
    image_size=768,
    crop_mode=True,
    save_results=True,
)
```

The key parameters you should watch immediately:

- `base_size=1024` (global normalization)
- `image_size=768` (crop size)
- `crop_mode=True` (multi-crop / local views)

- `prompt` mode (markdown conversion vs plain OCR)

Supported prompt modes

DeepSeek's own "main prompts" are the two canonical modes:

- **Document → markdown (layout-aware):** `<image>\n<|grounding|>Convert the document to markdown.`
- **Free OCR (no layout preservation):** `<image>\nFree OCR.`

You actually expand this into a more product-friendly list:

- document → markdown
- "OCR this image" (grounding)
- free OCR
- figure parsing
- general description
- locating a referenced snippet via `<|ref|>`…

I'd explicitly treat these as **separate tasks** with separate eval sets:

- *transcription accuracy* (Free OCR)
- *structure correctness* (document → markdown)
- *region grounding* (Locate `<|ref|>`…)

This is how you avoid shipping a single "OCR" endpoint that behaves unpredictably.

Running OCR2 on PDFs with Concurrency

If you're doing anything beyond a demo, you don't want "one Python call per page" with no throughput story.

Have a look at the vLLM setup and scripts for:

- streaming output for images: `run_dpsk_ocr2_image.py`
- concurrent PDF processing: `run_dpsk_ocr2_pdf.py`
- batch eval for benchmarks: `run_dpsk_ocr2_eval_batch.py`

They pin an environment approach that installs a specific vLLM wheel (`vllm-0.8.5+cu118`) plus FlashAttention 2, and they explicitly note you may hit version mismatches between vLLM and Transformers unless you manage the environment carefully.

Easy take here:

- Transformers path is for correctness and integration.
- vLLM path is for throughput and batching.
- what's likely to generalize (reading flow),
- where fine-tuning might help (domain layouts).

DeepSeek-OCR 2 vs PaddleOCR-VL (and PaddleOCR-VL-1.5)

This is the comparison I'd take most seriously for open-source deployments.

PaddleOCR-VL positions itself as:

- resource-efficient
- tailored for document parsing
- integrating a NaViT-style dynamic resolution visual encoder with an ERNIE-4.5–0.3B language model
- supporting **109 languages**,
- and handling complex elements like tables/formulas/charts.

The model card shows OmniDocBench v1.5 performance plots and explicitly calls out SOTA performance across multiple sub-dimensions (text/formula/table/reading order), with metrics sourced from multiple benchmarks.

Model Type	Methods	Parameters	Overall↑	Text^{Edit}↓	Formula^{CDM}↑	Table^{TEDS}↑	Table$^{TEDS\text{-}S}$↑	Reading Order^{Edit}↓
Pipeline Tools	Marker-1.8.2 [22]	-	71.30	0.206	76.66	57.88	71.17	0.250
	Mineru2-pipeline [23]	-	75.51	0.209	76.55	70.90	79.11	0.225
	PP-StructureV3 [20]	-	86.73	0.073	85.79	81.68	89.48	0.073
General VLMs	GPT-4o [7]	-	75.02	0.217	79.70	67.07	76.09	0.148
	InternVL3-76B [24]	76B	80.33	0.131	83.42	70.64	77.74	0.113
	InternVL3.5-241B [25]	241B	82.67	0.142	87.23	75.00	81.28	0.125
	GPT-5.2 [26]	-	85.50	0.123	86.11	82.66	87.35	0.099
	Qwen2.5-VL-72B [27]	72B	87.02	0.094	88.27	82.15	86.22	0.102
	Gemini-2.5 Pro [28]	-	88.03	0.075	85.82	85.71	90.29	0.097
	Qwen3-VL-235B-A22B-Instruct [6]	235B	89.15	0.069	88.14	86.21	90.55	0.068
	Gemini-3 Pro [15]	-	90.33	0.065	89.18	88.28	90.29	0.071
Specialized VLMs	Dolphin [3]	0.3B	74.67	0.125	67.85	68.70	77.77	0.124
	OCRFlux-3B [29]	3B	74.82	0.193	68.03	75.75	80.23	0.202
	Mistral OCR [30]	-	78.83	0.164	82.84	70.03	78.04	0.144
	POINTS-Reader [4]	3B	80.98	0.134	79.20	77.13	81.66	0.145
	olmOCR-7B [31]	7B	81.79	0.096	86.04	68.92	74.77	0.121
	Dolphin-1.5 [3]	0.3B	83.21	0.092	80.78	78.06	84.10	0.080
	MinerU2-VLM [23]	0.9B	85.56	0.078	80.95	83.54	87.66	0.086
	Nanonets-OCR-s [32]	3B	85.59	0.093	85.90	80.14	85.57	0.108
	MonkeyOCR-pro-1.2B [1]	1.9B	86.96	0.084	85.02	84.24	89.02	0.130
	Deepseek-OCR [10]	3B	87.01	0.073	83.37	84.97	88.80	0.086
	MonkeyOCR-3B [1]	3.7B	87.13	0.075	87.45	81.39	85.92	0.129
	dots.ocr [33]	3B	88.41	0.048	83.22	86.78	90.62	0.053
	MonkeyOCR-pro-3B [1]	3.7B	88.85	0.075	87.25	86.78	90.63	0.128
	MinerU2.5 [2]	1.2B	90.67	0.047	88.46	88.22	92.38	0.044
	PaddleOCR-VL [9]	0.9B	92.86	**0.035**	91.22	90.89	94.76	0.043
	PaddleOCR-VL-1.5	0.9B	**94.50**	**0.035**	**94.21**	**92.76**	**95.79**	**0.042**

Then PaddleOCR-VL-1.5 (an updated release) claims **94.5% accuracy on OmniDocBench v1.5** and introduces Real5-OmniDocBench for robustness under physical distortions (warping, skew, screen photography, illumination).

- If I care most about **layout reading flow** in complex documents, DeepSeek-OCR 2's "causal token reordering" is a uniquely targeted bet.
- If I care most about **multilingual coverage + efficiency in a pipeline** (and potentially robustness under real-world capture distortions), PaddleOCR-VL / 1.5 is extremely compelling on paper.

In reality I'd run both on a 200–500 page sample of my docs and score them on task-specific metrics.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Think Your Emails Are Safe? OpenClaw (Clawdbot) Proves Otherwise

More than 1800+ instances of OpenClaw on the public internet, over half were running with an unauthenticated gateway...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Kimi K2.5 + Agent Swarms Beat US AI Labs: Open Source vs OpenAI, Anthropic & Google

If you missed this week's Kimi K2.5 announcement, you have to read on.

medium.com

Qwen3-Max-Thinking Outperforms Claude Opus 4.5 and Gpt-5.2: The Panic Is Real

Alibaba's Qwen3 family of models has become a major alternative in the LLM ecosystem.

medium.com
#deepseek#ocr#gemini-3-pro#paddleocr#pdf-parsing

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel